# Introduction

This report outlines the development and customization of a DJ application, detailing the core functionalities, user interface customization, and the implementation of some new features inspired by professional DJ software.

# R1: Basic Program Functionality

## R1A: Audio File Loading

The application utilizes the JUCE framework to handle audio file loading. When a user initiates a file load via the GUI, either through a dialog box or drag-and-drop, the `DeckGUI` component interacts with the `DJAudioPlayer`. The `loadURL` method of `DJAudioPlayer` is called, which uses `AudioFormatManager` to ensure the file format is supported and then creates a `transportSource` that handles the playback of the audio file.

## R1B: Track Playback

For playback, `DJAudioPlayer` contains a `transportSource` which manages the play, stop, and pause functionalities. Commands from the GUI, like clicking the 'Play' or 'Stop' button, trigger these functionalities, allowing for real-time control over track playback.

## R1C: Track Mixing

The mixing functionality is powered by the `MixerAudioSource` in `DJAudioPlayer`. This mixer allows multiple audio sources to be combined. Each `DJAudioPlayer` instance (`player1`, `player2`) is connected to the mixer via `addInputSource`. This setup allows the application to control the mixing of two tracks.

## R1D: Speed Control

Speed adjustments are handled through a resampling process. The `DJAudioPlayer` uses a `ResamplingAudioSource` that wraps around the `transportSource`. Moving the speed control slider adjusts the resampling ratio, effectively speeding up or slowing down the track without changing the pitch.

These features are interconnected through event listeners and callback functions defined within the components, allowing for a responsive and interactive DJ application.

# R2: User Interface Customization

### R2A: GUI Layout

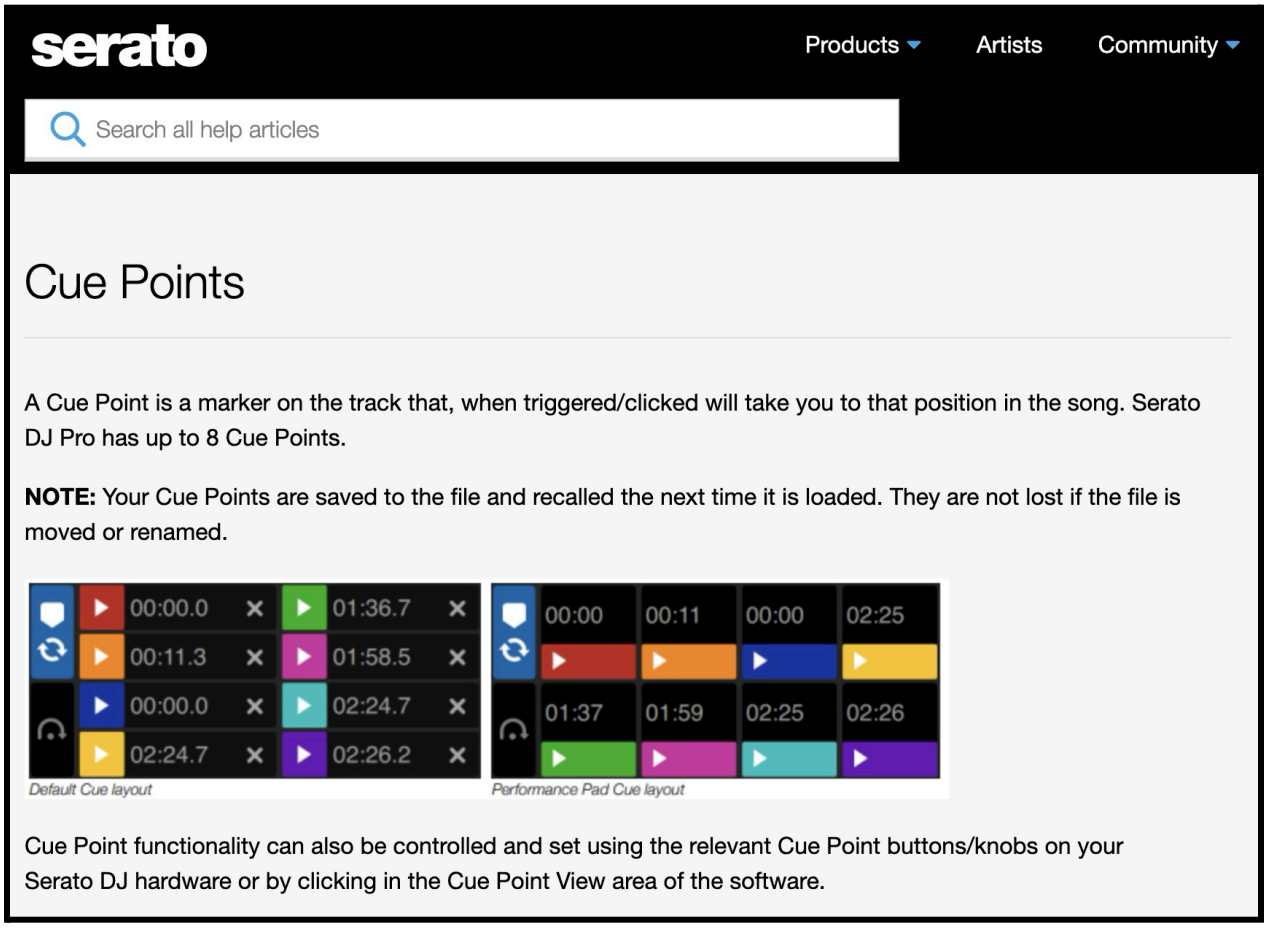
The graphical user interface was slightly redesigned to enhance usability and aesthetics. The layout was kept similar to the provided template to keep things intuitive while making aesthetic changes by updating the colors to enhance the look and feel. However, we have enhanced the User Interface and User Experience by developing a built-in Playlist feature that we will discuss later in this document.

### R2B: Event Listeners

New event listeners were integrated to respond to user interactions like setting cue points and jumping to them, which were not part of the original class demonstrations.

## R3: New Features Implementation

### Cue Point Management




Inspired by features in platforms like Serato DJ, cue point functionality was developed to allow DJs to mark and return to specific parts of a track. This was achieved by extending

`DJAudioPlayer` with methods for setting and recalling cues, and updating `DeckGUI` to include cue management buttons.

### Playlist Component

The Playlist Component, an original feature, allows for dynamic playlist management within the application. It supports loading playlists from an XML file and interacting with the track list directly to load music into the decks.

## Engineering Details for Playlist Component

### Class Structure

- **PlaylistComponent**: Inherits from JUCE's `Component`, `TableListBoxModel`, and `Button::Listener`. It manages the playlist UI and interactions.
- **TableListBox**: Used for displaying track titles and providing interactive buttons for adding tracks to players.
- **TextButton**: Interactive buttons for each track to load them into the designated player.

### Implementation Details

- **Dynamic Path Handling**: Depending on the operating system, the component calculates the correct path to the project directory to ensure that the playlist XML and track files are accurately located. This is handled by preprocessor directives (`JUCE_WINDOWS`, `JUCE_MAC`) to adapt the file path generation based on the platform.
- **XML Parsing**: The component reads from an XML file to populate the track list. It uses JUCE's `XmlDocument` and `XmlElement` classes to parse the XML structure and extract track information.
- **Interaction with DeckGUI**: PlaylistComponent communicates with two instances of DeckGUI to load tracks. This is facilitated by methods `setDeckGUI1` and `setDeckGUI2`, which link the playlist with the audio players.
- **Event Handling**: Implements the `Button::Listener` interface to respond to user interactions, allowing tracks to be loaded into the decks through its UI.

### Object-Oriented Features

- **Encapsulation**: All internal states of the playlist, such as track titles and paths, are kept private, with methods provided for specific interactions.
- **Inheritance**: By extending from JUCE’s base classes, PlaylistComponent leverages polymorphism for event handling and UI management.
- **Composition**: Utilizes `TableListBox` and `TextButton` components to build a complex UI.

# Conclusion

The enhancements and new features added to the DJ application not only improve its functionality but also align it with professional standards, offering tools that are both practical and desirable.